

Action is a record with

required fields:

key: string;

optional fields:

title: string;

description: string;

robotActionKey: string;

payload: record like (all fields optional)

```
{
  preset: (values directly defined in the setup)
  {
    field (string): value (any primitives, array of primitives, record),
    ...
  }
  cached: (values taken from the game cache)
  {
    field (string): cache field (string),
    ...
  }
  computed: (values calculated during the game based on the state and cache of the game)
  {
    field (string): Computed Payload Field
  }
}
```

skipCondition: [same as for Action Groups](#)

saveValueInCache: to save some values in the game cache;

```
Array of {
  name: string; (cache field name)
  value: any value or Computed Payload Field
}
```

postHandler: string ([handler](#) name)

```
OR record {
  handler: string, (handler name)
  params: Computed payload field params
};
```

([handler](#) name to process the result of the action. If the handler is specified, it accepts the result of the action, processes it and saves it to the cache as lastActionResult, else the result of action is directly saved to the cache as lastActionResult.)

Action sounds

To play sounds during the action, you must add the following fields to the payload:

sound.list: Array of strings (sounds aliases from [gameInitOption](#). For delay between sounds use `delay.number`, where number is the duration in milliseconds.)

sounds.waitForSoundsEnd: boolean (if true - the action does not end until all sounds have been played)

playList.[number]: Array of strings (userIds which should have a sound with a sequence number from the list of sounds. If it is necessary to play all sounds from sounds.lists for the same set of players, in order not to write playList.1, playList.2 and so on, use playList.0.)

example of usage:

```
"payload": {
  "preset": {
    "sounds.waitForSoundEnd": true,
    "sounds.list": [
      "voices.ding",
      "voices.mafia_action",
      "voices.vigilante_action",
      "soundboard.ding"
    ]
  },
  "computed": {
    "playList.1": {
      "selector": "allPlayers"
    },
    "playList.2": {
      "selector": "allPlayers"
    },
    "playList.3": {
      "selector": "allPlayers"
    },
    "playList.4": {
      "selector": "allPlayers"
    }
  }
}
```

1. Core actions

- 1.1. [createNotification](#)
- 1.2. [createConfirmation](#)
- 1.3 [emptyAction](#)
- 1.4 [highlightPlayers](#)
- 1.5 [removeHighlight](#)
- 1.6 [mutePlayers](#)
- 1.7 [unmutePlayers](#)
- 1.8 [changeLayout](#)
- 1.9 [setImagesRow](#)
- 1.10 [changeImageInRow](#)
- 1.11 [changeBackground](#)

- 1.12 [createConversationGroup](#)
- 1.13 [destroyConversationGroup](#)
- 1.14 [destroyConversationGroups](#)
- 1.15 [createPuzzgrid](#)
- 1.16 [animateBox](#)
- 1.17 [createMessage](#)
- 1.18 [removeWidget](#)
- 1.19 [restoreWidget](#)
- 1.20 [createInput](#)
- 1.21. [showTeam](#)
- 1.22. [showRole](#)
- 1.23. [showFakeTeam](#)
- 1.24. [showTrueRoles](#)
- 1.25. [showFakeRole](#)
- 1.26. [createTeamsConversationGroups](#)
- 1.27. [muteTeam](#)
- 1.28. [setRole](#)
- 1.29 [removeAllHighlights](#)
- 1.30 [orderByTeam](#)
- 1.31 [hideInvisiblePlayers](#)
- 1.32 [showAllPlayersHands](#)
- 1.33 [hideRole](#)
- 1.34 [drawArrows](#)
- 1.35 [setGameEndSound](#)
- 1.36 [roleConfirmation](#)
- 1.37 [createMissionHistory](#)
- 1.38 [updateMissionHistory](#)
- 1.39 [createAdvertisement](#)
- 1.40 [create Drawing](#)
- 1.41 [hideAllPlayersHands](#)
- 1.42 [restoreMuteStatus](#)
- 1.43 [takeWidgetScreenshot](#)
- 1.44 [showPlayersHands](#)
- 1.45 [hidePlayersHands](#)
- 1.46 [startClip](#)
- 1.47 [endClip](#)
- 1.48 [createOneWordClue](#)

2. Modules actions

'votes'

- 1. [createVote](#)
- 2. [createMixVote](#)
- 3. [getTriviaQuestions](#)

'playerScore'

1. [showScore](#)
2. [updateScore](#)
3. [removeScore](#)
4. [orderByScore](#)

'deadPlayer'

1. [killPlayer](#)
2. [killPlayers](#)
3. [makeAllPlayersAlive](#)
4. [createIndividualConversationGroups](#)

'cards'

1. [createDeck](#)
2. [createPublicDeck](#)
3. [createPlayersDecks](#)
4. [createCustomDeck](#)
5. [shuffleDeck](#)
6. [dealDeck](#)
7. [playCards](#)
8. [moveCards](#)
9. [recallCards](#)
10. [discard](#)
11. [sortDeck](#)
12. [showHand](#)
13. [takeFivePlaceCards](#)
14. [setCardsPile](#)
15. [createGenericCardWidget](#)
16. [createVideoBoxDecks](#)
17. [noThanksPlaceCards](#)
18. [selectCentralWidgetDeck](#)
19. [flipOverTopCard](#)
20. [createCard](#)
21. [createTextCards](#)
22. [setDeckInspectors](#)
23. [setInspectDeck](#)
24. [setDeckLabels](#)
25. [setDeckLabel](#)

'grids'

1. [setMasterClue](#)
2. [teamTurn](#)
3. [createCodenames](#)

'waw'

1. [createWawBoard](#)
2. [wawBetting](#)

3. [wawReveals](#)
4. [createWaw](#)

3. Application actions.

'mafia'

1. [setDay](#)
2. [setNight](#)

createNotification

creates a widget with the for the selected users and a record in the history of the game.

key: 'createNotification'

payload:

required fields:

to: Array of strings; (user Ids)

one of the 'text' or 'header' fields is required:

text: string;

header: string;

optional fields:

duration: number; (in seconds, default 5)

isAnnounceOnly: boolean; (if 'true' creates a record in the history only)

isBlocking: boolean; (if 'true' blocks further actions until the time set in duration elapses)

image: string;

suppressHistory: boolean; (if 'true' the history item for that action is not shown);

images: array of strings (images aliases or URLs)

backgroundColor: string; (CSS value for widget)

textColor: string; (CSS value for widget)

borderColor: string; (CSS value for widget)

createConfirmation

creates a widget with text for actors and waits for a specified number of confirmations or the end of the timer

key: 'createConfirmation'

payload:

required fields:

actors: Array of strings; (user Ids)

text: string;

passCriterion: string; ('all', 'two-thirds', 'majority')

optional fields:

timer: number; (in seconds)

type: string; ('my_role', 'recorder')

userSpecifiedText: Record<string, string>; (userId -> custom text)

header: string;

allowRejection: boolean;

image: string;

backgroundColor: string; (CSS value for widget)

textColor: string; (CSS value for widget)

borderColor: string; (CSS value for widget)

emptyAction

action without a payload and any specific activity. Is mainly used to save values in cache.

key: 'emptyAction'

payload:

optional fields:

delay: number (in seconds')

highlightPlayers

create color highlights for player's videobox.

key: 'highlightPlayers'

payload:

required fields:

listOfPlayers: Array of strings; (user Ids)

color: string; (as CSS value)

removeHighlight

remove highlight

key: 'removeHighlight'

payload:

required fields:

id: string; (id of created highlight)

removeAllHighlights

remove all highlights

key: 'removeAllHighlights'

orderByTeam

order players by teams

key: 'orderByTeam'

showAllPlayersHands

makes your cards become visible from the bottom toolbar, even though you can't play them

key: 'showAllPlayersHands'

hideAllPlayersHands

makes your cards become hidden from the bottom toolbar

key: 'hideAllPlayersHands'

setHideAllPlayersHands

makes your cards become hidden from the bottom toolbar

key: 'hideAllPlayersHands'

showPlayersHands

Shows the player's hand

key: 'showPlayersHands'

payload:

required fields:

userids: Array of strings

hidePlayersHands

Hide the player's hand

key: 'hideAllPlayersHands'

payload:

required fields:

userids: Array of strings

hideInvisiblePlayers

change hideInvisiblePlayers setting

key: 'hidePlayersHands'

payload:

required fields:

hide: boolean

mutePlayers

change mute status for players list (except for those with status of NO_DEVICE)

key: 'mutePlayers'

payload:

required fields:

players: Array of strings; (userIds)

optional fields:

audio: string (one of 'MUTED', 'MUTED_BY_ADMIN', 'MUTED_BY_GAME')

video: string (one of 'MUTED', 'MUTED_BY_ADMIN', 'MUTED_BY_GAME')

unmutePlayers

change mute status to 'MUTED' for players list (except for those with the status of NO_DEVICE)

key: 'unmutePlayers'

payload:

required fields:

players: Array of strings; (userIds)

optional fields:

audio: boolean;

video: boolean;

restoreMuteStatus

restores the mute status of all players to the last saved mute status

key: 'restoreMuteStatus'

takeWidgetScreenshot

take a screenshot of the central widget. The screenshot gets saved to a folder in Clouldinary

key: 'takeWidgetScreenshot'

payload:

required fields:

title: string;

changeLayout

key: 'changeLayout'

payload:

required fields:

type: string; ('ANNULUS', 'HIGHLIGHT', 'GRID')

optional fields:

top: Array of strings; (user ids)

middle: Array of strings; (user ids)

bottom: Array of strings; (user ids)

percent: number; (If the field is missing, then auto percent)

direction: string; ('HORIZONTAL', 'VERTICAL') required for 'HIGHLIGHT'

setImagesRow

key: 'setImagesRow'

payload:

required fields

images: string[]; (image names)

optional fields:

maxHeight: number;

changeImageInRow

key: 'changeImageInRow'

payload:

index: number; (position in the row)

image: string; (image name)

changeBackground

key: 'changeBackground'

payload:

image: string; (url or image name)

createConversationGroup

key: 'createConversationGroup'

payload:

required fields

members: Array of strings; (user ids)

indicator: string

optional fields (but there must be at least one of them):

audio: boolean;

video: boolean;

private: boolean; (If 'true' - do not display any indicator of who is in which group. You only see the indicators for people in your group)

isVideoForAll: boolean; (If 'true' - all users can see this group members video)

destroyConversationGroup

key: 'destroyConversationGroup'

payload:

id: string;

destroyConversationGroups

key: 'destroyConversationGroups'

payload:

optional fields:

ids: string; (If the field isn't present ALL groups are destroyed)

createPuzzgrid

creates a widget with PuzzGrid game

key: 'createPuzzgrid'

payload:

required fields:

actors: Array of strings; (user ids, who can interact with the PuzzGrid, i.e. who can click on the words and the Reveal buttons)

optional fields:

id: string; (name id, if not - the game is selected at random)

time: number; (in seconds. how long the game should last. If no timer is specified, the PuzzGrid is untimed)

private: boolean (if 'true', only the actors and spectators can see the PuzzGrid, else everyone sees it)

allowSkipping: boolean;

minDifficulty: number; (the minimum acceptable difficulty between 1 and 3, default 1)

maxDifficulty: number; (the maximum acceptable difficulty between 1 and 3, default 3)

returns result object:

```
{  
  timeTakenSec: number;  
  attempts: number;
```

```
    successfulAttempts: number;  
    solved: boolean;  
}
```

animateBox

key: 'animateBox'

payload:

required fields:

userIds: array of strings;

animation: string (animation alias);

optional fields:

delay: number (set the delay before the next action plays in seconds. Default 3)

removeWidget

key: 'removeWidget'

payload:

required fields:

id: string;

restoreWidget

key: 'restoreWidget'

payload:

required fields:

id: string;

createMessage

creates chat message

key: 'createMessage'

payload:

required fields:

text: string;

sender: string; (sender userId)

optional fields:

chatId: string; (default "common")

to: string; (userId. If the field isn't present ALL users receive message)

setDay

change audio and video statuses from MUTED_BY_ADMIN to MUTED for all players

key: 'setDay'

setNight

change audio and video statuses to MUTED_BY_ADMIN for all players

key: 'setNight'

createMissionHistory

key: 'createMissionHistory'

payload:

required fields:

type: string; ('avalon', 'secret_hitler', 'cards')

title: string;

players: string[]; (user ids)

example of usage:

```
{
  "key": "createMissionHistory",
  "payload": {
    "preset": {
      "type": "avalon",
      "title": "Avalon"
    },
    "computed": {
      "players": {
        "selector": "allPlayers"
      }
    }
  }
}
```

updateMissionHistory

key: 'updateMissionHistory'

payload:

required fields:

missionNumber: number;

proposalRound: number;

missionProposal: string[];
proposer: string OR number; (userId OR index in **players** from createMissionHistory)
answersVoters: Record<string, string[]> (IVote['result']['answersVoters']);
 the vote positive answer must be:
 'ACCEPT' for 'avalon' createMissionHistory type
 'JA!' for 'secret_hitler'
 'ACCEPT' for 'cards'

OR

isMissionPassed: boolean;

examples of usage:

```
{
  "key": "updateMissionHistory",
  "payload": {
    "cached": {
      "missionNumber": "missionNumber",
      "proposalRound": "proposalRound",
      "missionProposal": "missionProposal",
      "proposer": "proposer",
      "answersVoters": "lastActionResult.answersVoters"
    }
  }
}
{
  "key": "updateMissionHistory",
  "payload": {
    "cached": {
      "isMissionPassed": "missionPassed"
    }
  }
}
```

createAdvertisement

creates an advertising notification

key: 'createAdvertisement',

payload:

required fields:

preset:

header: string,

text: string (may contain link, example: 'text text text link text<a/>text')

createInput

create a widget for input.

key: 'createInput'

payload:

required fields:

title: string;

type: string; ('number', 'text')

question: string;

terminationCondition: string; (action will end when:

'time', - **duration** runs out

'get_all_values', - all actors have set value

'get_majority', - 50% + 1 of actors number have set value

'get_two_thirds' - $\frac{2}{3}$ of actors number have set value

scope: string: ('team' or 'player' - type of actors)

actors: Array of strings; (user Ids OR team Ids depending on **scope**)

optional fields:

defaultValue: number | string; (default: 0 or "" depending on **type**)

duration: number; (in seconds, if the value is set action will be interrupted after the duration expires.

Needed when terminationCondition='time')

private: boolean; (if 'true' input widget is visible for actors only)

showResult: boolean; (if 'true' the user input value is shown over their face for ShowResultDuration seconds)

showResultDuration: number; (in seconds, the user input value will be displayed for the set number of seconds. Needed when **showResult=true**)

allowSkipping: boolean; (if 'true' allows the player to skip the action)

checkIfValid: boolean; (If 'true' then real English words are allowed)

allowResubmit: boolean; (if 'true' the actor can type something else and resubmit, overriding their last submission)

checkIfValid: boolean; (if 'true', only valid English words are allowed)

checkIfSingle: boolean; (if 'true', only one word is allowed)

overrideSpellCheck: boolean; (if 'true', allows resubmitting the same word without re-checking)

revealResults: boolean; (if 'true' all the users submissions is shown over their face when the action terminates)

suppressHistory: boolean; (if 'true' the history item for that action is not shown, default 'false');

maxCount: number; (cap the number of characters for text input)

backgroundColor: string; (CSS value for widget)

textColor: string; (CSS value for widget)

borderColor: string; (CSS value for widget)

lowerCase: boolean; (forces the player's answer to be lowercase)

options: Record<string, any> (object with arbitrary fields passed directly to the frontend. Ex.(for ROLF game)

options.startLetters: ["C"],

options.restrictedWords: ["Camel"],

options.restrictedLengths = [7]

)

vertical: boolean;

checkIfValid: boolean; whether to check a word for presence in the dictionary of real words

overrideSpellCheck: boolean; whether skip word checking when resubmitting

checkIfSingle: boolean; need to check that only one word is sent

image: string | string[] ; string[] - there should be only two images in this array

returns result object:

```
{  
  [id]: input value
```

}

drawArrows

key: 'drawArrows'

payload:

required fields:

arrows: Record { actor: targets, ...} , where
actor: string () player id - from whom the arrow is drawn
targets: string[] - players ids, to whom the arrow is drawn

optional fields:

duration: number; (in seconds. Default: 1 sec.)

setGameEndSound

key: 'setGameEndSound'

payload:

required fields:

sound: string; (URI or alias from **soundboard.default** library)

roleConfirmation

key: 'roleConfirmation'

createDrawing

create a widget for drawing.

key: 'createDrawing'

payload:

required fields:

actors: Array of strings; (user ids who can draw)

question: string;

terminationCondition: string; (action will end when:
'time', - **duration** runs out
'get_all', - all actors have finished drawing)

optional fields:

defaultImage: string;

duration: number; (in seconds, if the value is set action will be interrupted after the duration expires.

Needed when terminationCondition='time')

private: boolean; (If true, only the actors can see the whiteboard widget. false by default)

colors: array of string; (List of colors can be used in the drawing. If not specified, use a standard color palette)
capture: boolean; (If true, takes a snapshot of the whiteboard when the action terminates, and saves it to Cloudinary. false by default)

returns result object (if **capture = true**):

```
{  
  url: string  
}
```

createDrawing

create a widget for drawing.

key: 'createDrawing'

payload:

required fields:

actors: Array of strings; (user Ids who can draw)

question: string;

terminationCondition: string; (action will end when:

'time', - **duration** runs out

'get_all', - all actors have finished drawing

optional fields:

defaultImage: string;

duration: number; (in seconds, if the value is set action will be interrupted after the duration expires.

Needed when terminationCondition='time')

private: boolean; (If true, only the actors can see the whiteboard widget. false by default)

colors: array of string; (List of colors can be used in the drawing. If not specified, use a standard color palette)

capture: boolean; (If true, takes a snapshot of the whiteboard when the action terminates, and saves it to Cloudinary. false by default)

returns result object (if **capture = true**):

```
{  
  url: string  
}
```

startClip

starts recording a clip.

key: 'startClip'

payload:

required fields:

title: string;

endClip

stops recording a clip.

key: 'endClip'

createOneWordClue

creates a one-word clue input for a single actor.

key: 'createOneWordClue'

payload:

required fields:

actor: string;

duration: number; (in seconds)

restrictedWords: string[];

optional fields:

backgroundColor: string; (CSS value for widget)

textColor: string; (CSS value for widget)

borderColor: string; (CSS value for widget)